

Efficient Pipeline Parallelism for Reinforcement Learning

Faiz Sameer Ahmed

MS Computer Science - Spring 2026

Advisor

Dr. Genya Ishigaki

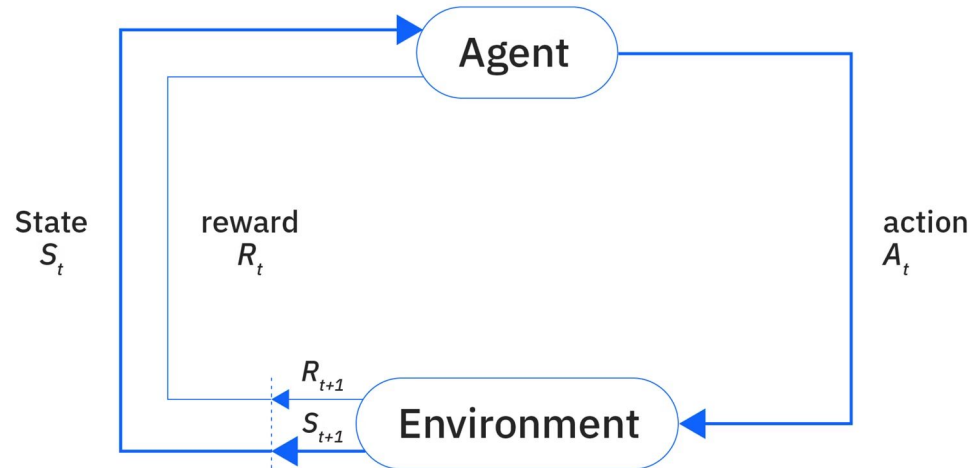
Committee Members

Dr. Jelena Gligorijevic

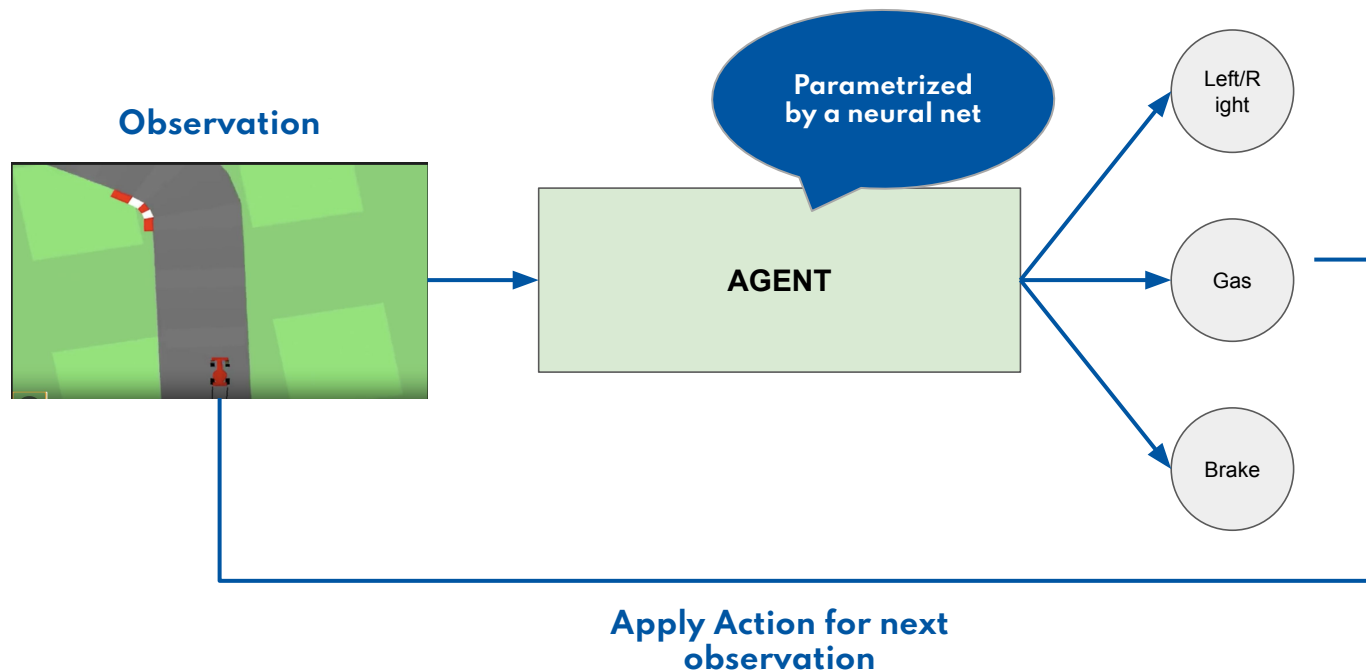
Dr. Fabio Di Troia

May 2026

Reinforcement Learning - learns through trial and error by interacting with the environment



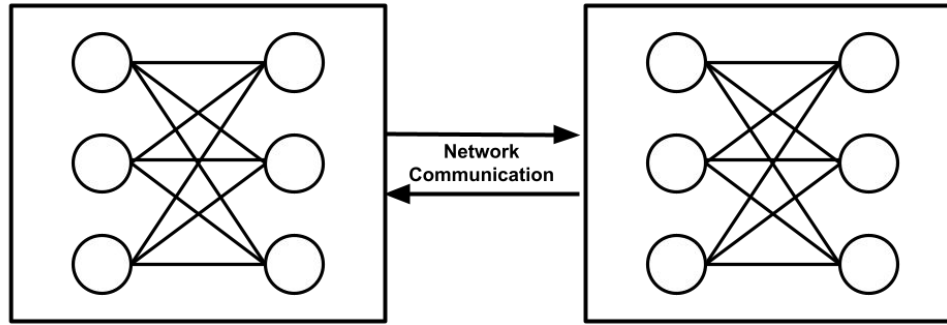
Zooming into the Agent





In resource constrained environments (eg. IoT devices), the agent cannot run on a single device.

So how do we train RL in scenarios where the environment is resource constrained?



- Pipeline parallelism [1] - the model is split across machines
- During training, both machines need to talk to each other every minibatch, every epoch
- Forward pass: Machine 0 sends activations to Machine 1
- Backward pass: Machine 1 sends gradients back

- **On-Policy RL training has two phases:**
 - **Rollout - collect data by interacting with the environment**
 - **Training - update the model using the collected data**
- **During training, the model is updated over multiple epochs on the same data which requires activations and gradients to cross the network**
- **More epochs = more network usage**



Research Statement

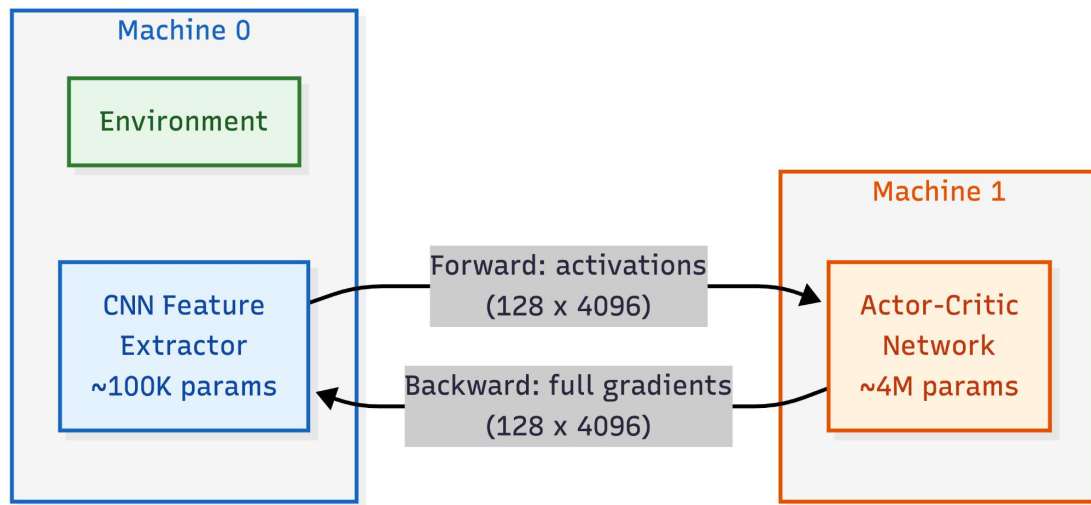
Reduce network data transfer during on-policy RL training w/ Pipeline Parallelism, while keeping performance similar to the baseline.

Related Work

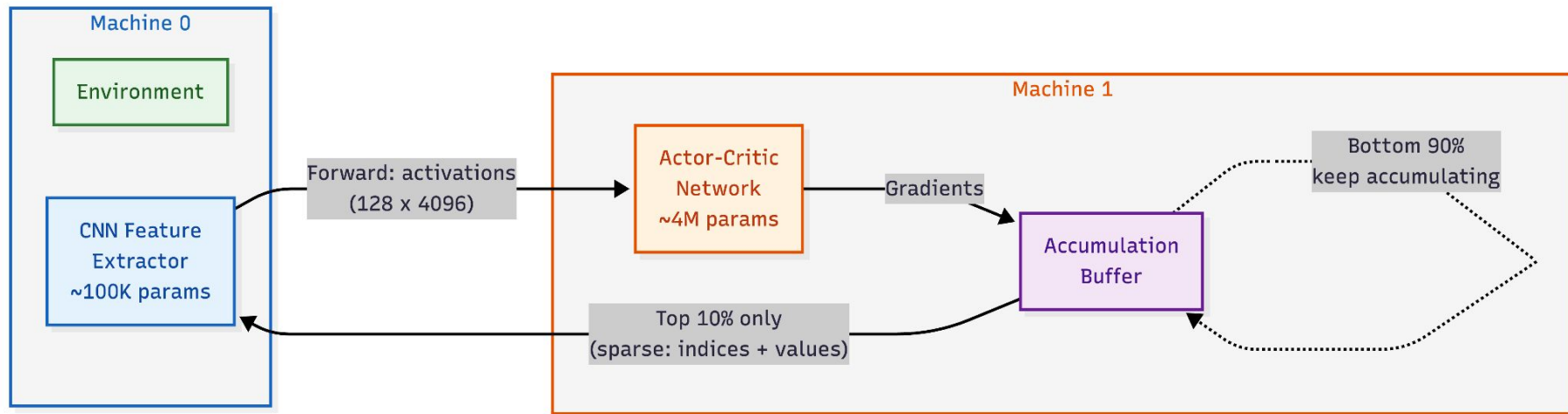
- **Deep Gradient Compression [2]** sparsifies gradients in data-parallel supervised learning by sending only the largest gradients and accumulating the rest locally
- **Adaptive Communication Skipping [3]** reduces communication in distributed RL by skipping entire gradient exchange rounds when updates are similar to previous ones
- **Both approaches target data-parallel setups where each worker holds a full copy of the model**
- **Our work applies gradient sparsification to pipeline-parallel RL, where each machine holds a different part of the model, with local gradient accumulation and exponential buffer decay to handle staleness**

[2] Y. Lin, S. Han, H. Mao, Y. Wang, W. Dally. "Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training." ICLR, 2018.
[3] T. Chen, K. Zhang, G. Giannakis, T. Basar. "Communication-Efficient Policy Gradient Methods for Distributed Reinforcement Learning." IEEE Trans. Control of Network Systems, 2021.

Standard Pipeline: Send Everything, Every Time



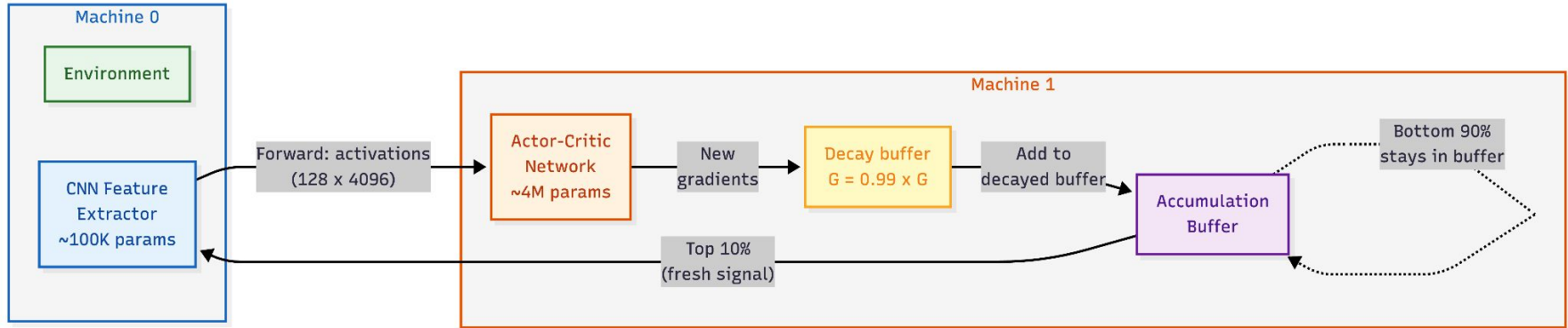
Our Method: Accumulate and Sparsify Gradient Transfer



- **Most gradients are not impactful during an update as seen in DGC [4]**
- **Only gradients above the 90th percentile magnitude are sent; the rest stay in the buffer**
- **Unsent gradients keep accumulating until they become large enough to cross the threshold**

Buffer Decay

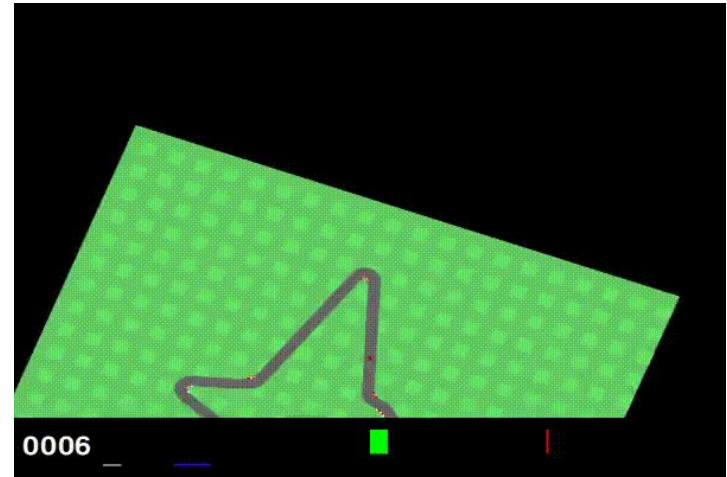
Old gradients pile up and become stale causing delayed harmful updates. We decay the accumulated buffer during each update



Warm Start

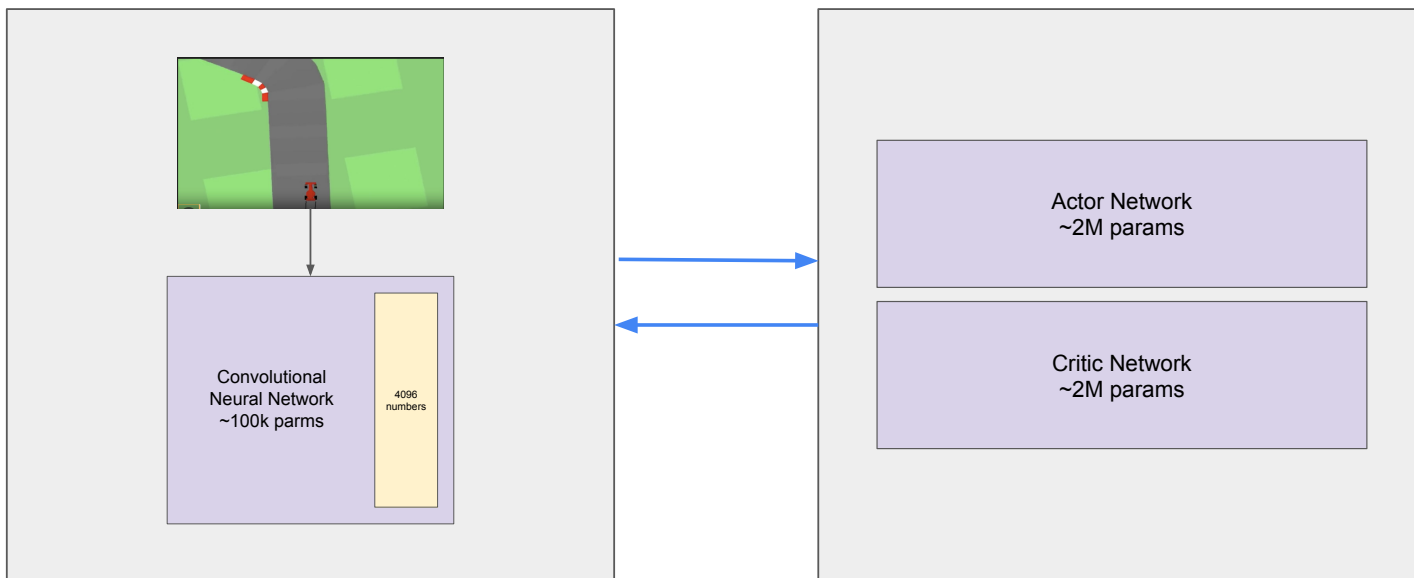
- **First N steps: send full gradients with no compression to let the model develop stable initial weights**
- **After warm start: gradient accumulation with sparsification and buffer decay activates**
- **Early gradients are noisy and the model needs every update to stabilize before we start filtering**

Train a self driving AI
(CarRacing-v3 Gymnasium Env)
using the same hyperparams
using different methods and
compare the bandwidth
consumption and performance

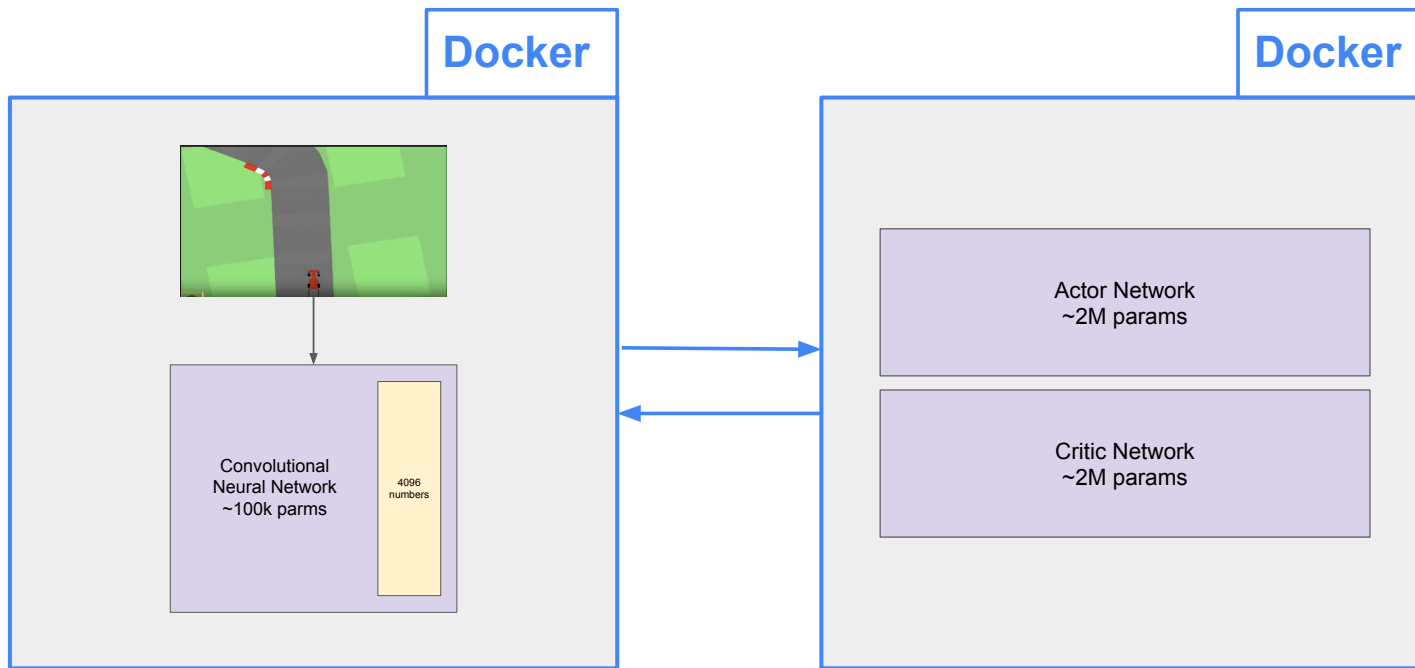


We use PPO [5] RL algorithm with an actor-critic architecture, split across two machines:

- Machine 0: CNN feature extractor ~100k params, last layer shape 4096
- Machine 1: Actor and Critic networks ~2M params each



We simulate the 2 machines using docker and measure the network coming in and out of each container



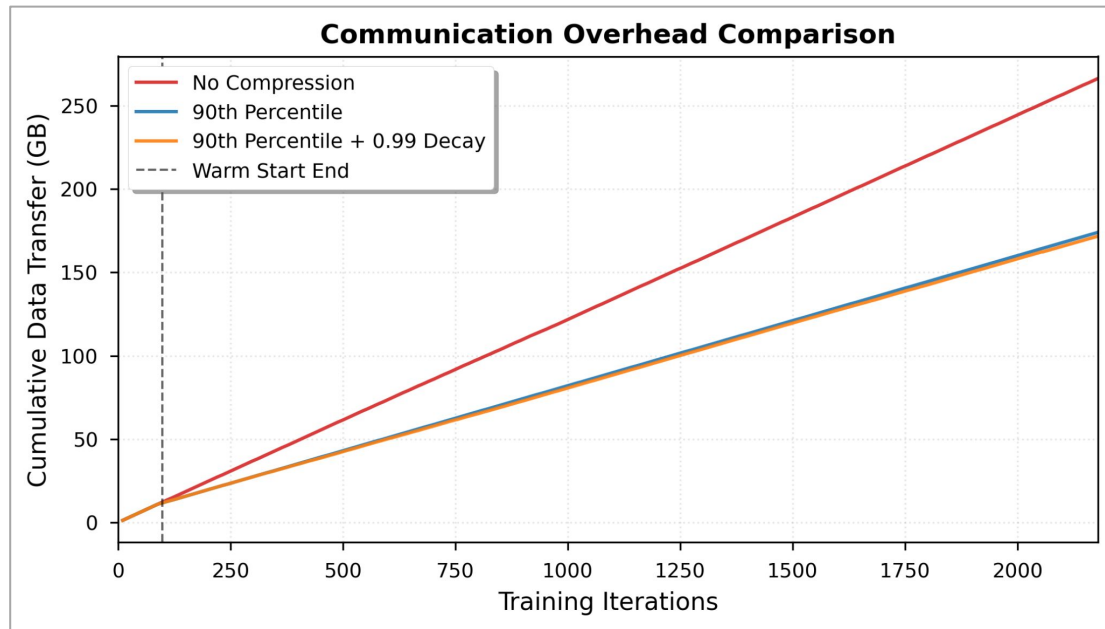
Gradient Sparsification was tested across various percentile thresholds and compared

Parameter	Values Tested
Percentile threshold	90th, 95th, 99th
Buffer decay (λ)	1.0 (no decay), 0.99, 0.90
Random seeds	2 seeds per config
Training budget	1M timesteps
Minibatches x Epochs	32 x 10 = 320 per iteration
Warm start	30K steps

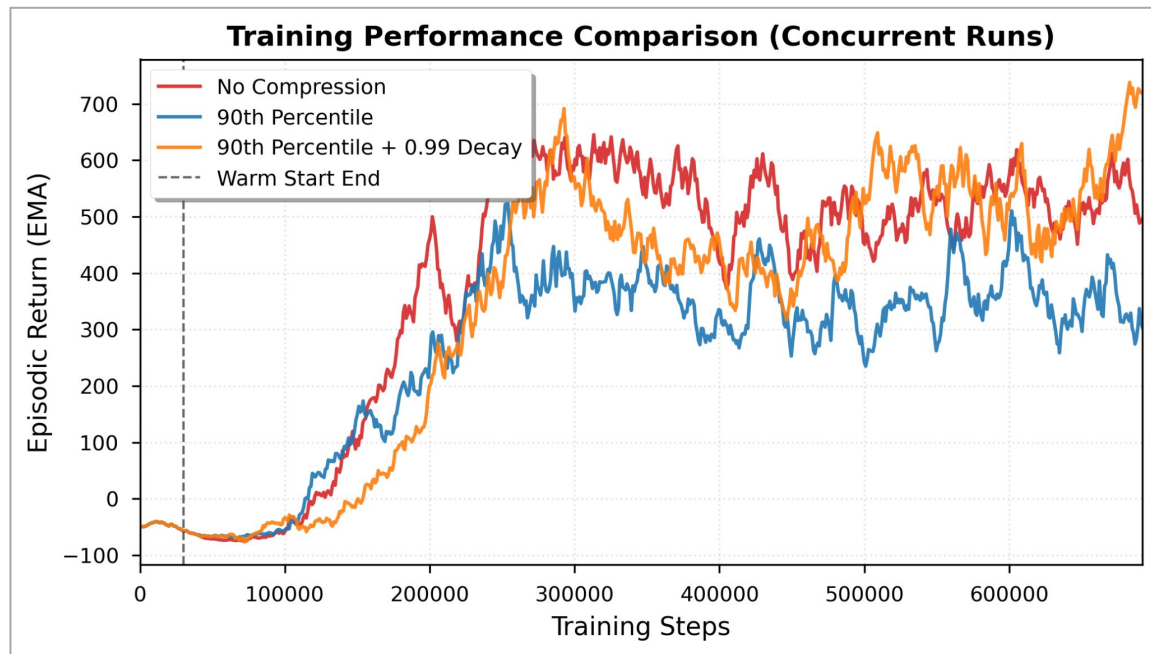
Return Evaluation

- Each method's last checkpoint is evaluated over 100 independent episodes on randomly generated tracks
- No further training is performed during evaluation, the further the car goes, higher the return
- We report mean, median, and interquartile range (IQR) to capture both central tendency and variance

**~30% reduction in data
on the wire vs the
baseline**

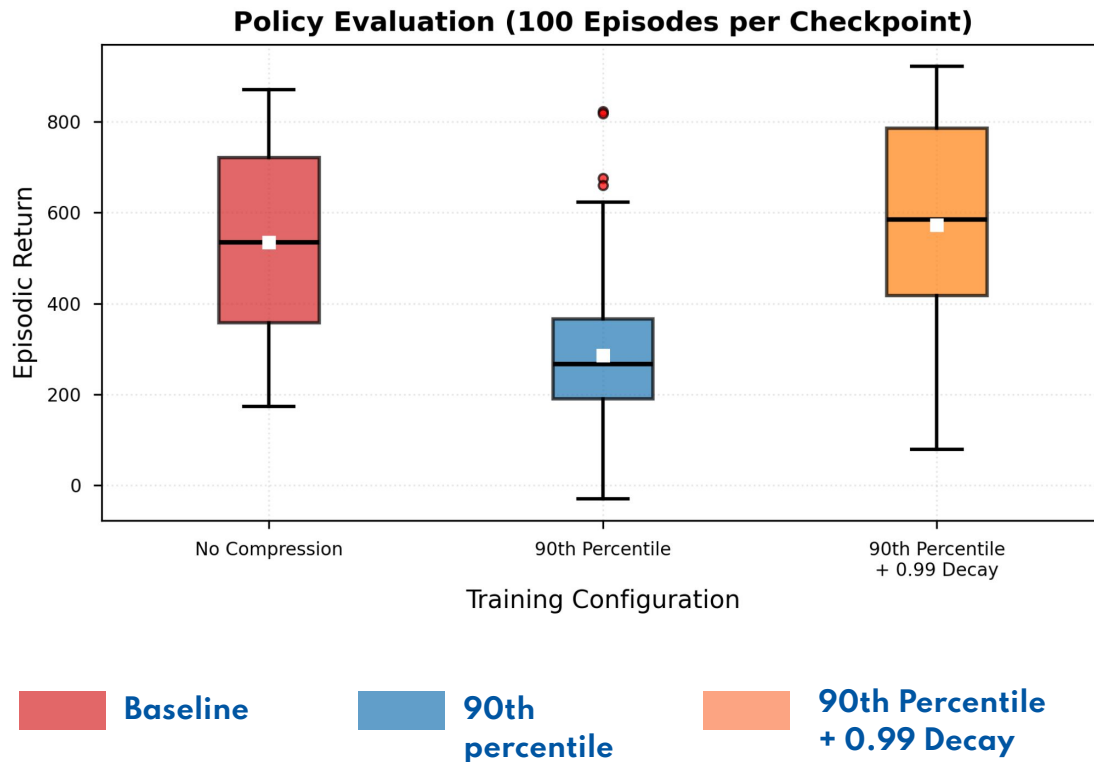


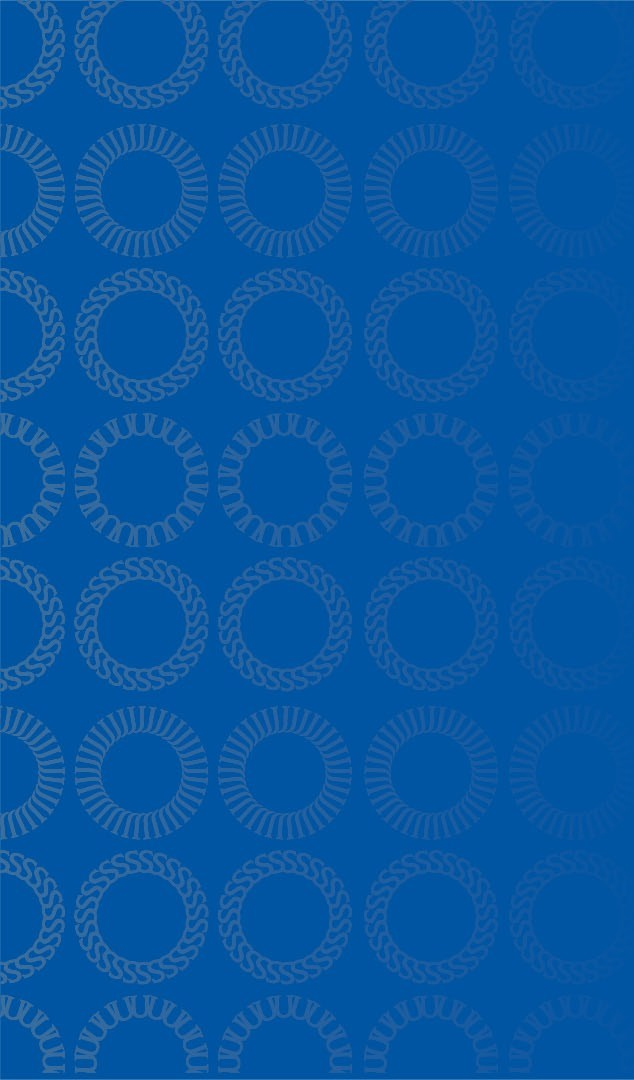
Training is stable and follows the baseline with similar episodic returns



Return Evaluation

Performs as well as the baseline (tested with multiple seeds)





Thank you!